Pypomp: Inference for partially observed Markov process models in Python with JAX
DRAFT IN PROGRESS

Aaron J. Abkemeier[*1] aaronabk@umich.edu
Jun Chen[*2] chenjunc@umich.edu
Kevin Tan[3] kevtan@wharton.upenn.edu
Jesse Wheeler[4] jessewheeler@isu.edu
Bo Yang[5] ybb@umich.edu
Kunyang He[1] kunyanghe@umich.edu
Jonathan Terhorst[1] jonth@umich.edu
Aaron A. King
Edward L. Ionides[1] ionides@umich.edu
[*]These authors contributed equally

[1]*University of Michigan, Department of Statistics;* [2]*Duke University, Department of Statistics;* [3]*University of Pennsylvania, Department of Statistics;* [4]*Idaho State University, Department of Mathematics;* [5]*Columbia University, Department of Marketing*

Editor:

Abstract

Model development and parameter estimation for non-Gaussian partially observed Markov process (POMP) mechanistic models are fundamental challenges in a variety of fields including epidemiology, ecology, and finance. This task is often hindered by high computational demands. We introduce Pypomp, a high-performance Python package for statistical inference using POMP models. Designed for complex stochastic dynamic systems, Pypomp implements plug-and-play, likelihood-based algorithms including sequential Monte Carlo and iterated filtering. By leveraging GPU acceleration, Pypomp significantly outperforms its predecessor, the R package pomp [EL: MAYBE DON'T MENTION R-POMP IN THE ABSTRACT?]. Models that previously required months to fit now complete in days, enabling statistical inference on previously intractable models. Furthermore, Pypomp uses JAX to perform gradient descent on differentiable particle filters, achieving likelihood maximization beyond the practical limits of iterated filtering alone. The package provides a comprehensive interface that streamlines model construction, fitting, and analysis, allowing practitioners to develop and evaluate complex models with minimal implementation overhead.

Keywords: pomp, JAX, GPU acceleration, likelihood-based inference

1 Introduction

Partially Observable Markov Process (POMP) models, also known as state-space models or hidden Markov models, provide a flexible mechanistic framework for modeling time-series dynamic systems where the underlying state process is not directly observed. Methods such as particle filtering (Arulampalam et al., 2002) and iterated filtering (Ionides et al., 2006, 2015) have been developed to perform maximum likelihood estimation and evaluation for POMP models. A defining strength of these methods is that they are performed by simulating from the process model instead of evaluating exact transition probabilities (a property known as plug-and-play (Bretó et al., 2009)), which makes them suitable for examining a wide range of models unconstrained by a need for mathematical convenience. Remarkably, algorithms based on particle filtering permit statistically efficient, likelihood-based inference in various practical situations even when the likelihood can be approximated only by Monte Carlo methods. Consequently, POMP models find extensive application in epidemiology (Mietchen et al., 2024; Fox et al., 2022; Wen et al., 2024), ecology (Auger-Méthé et al., 2021; Marino et al., 2019; Blackwood et al., 2013), finance (Bretó, 2014), and other domains where mechanistic modeling is desired.

The R-pomp ecosystem in R provides a basis for POMP modeling of several types [ELI: I'VE TRIED REWORKING THIS. SEE IF YOU LIKE IT. MAYBE THERE'S MORE TO DO, BECAUSE THE FOCUS MUST BE ON PYPOMP NOT R-POMP]. The core of R-pomp is the `pomp` package (**?**) designed for standard POMP models, while its extension packages `panelPomp`, `spatPomp`, and `phyloPomp` add capabilities for panel, spatio-temporal, and phylodynamic data analysis, respectively (Bretó et al., 2025; Asfaw et al., 2024; King et al., 2022).

While powerful, statistical inference for POMP models poses substantial computational challenges, which existing packages struggle with due to their lack of support for GPU acceleration and automatic differentiation. To address this, we introduce the package Pypomp (Abkemeier et al., 2026), a JAX-based (Bradbury et al., 2018) Python implementation of core R-pomp algorithms [USUALLY, SOFTWARE PACKAGES DON'T DEFINE ALGORITHMS]. [AA: Need to change "90 times faster" to something more specific after we more rigorously compare pypomp with pomp. In addition, I think we should include the @korevaar2020 data set fit in this paper, as it provides perhaps the strongest motivation for developing pypomp.] By leveraging GPU acceleration, the package allows users to run methods up to 90 times faster than the CPU-based alternatives, enabling plug-and-play likelihood-based inference on data sets that would otherwise be intractably large. This is demonstrated in Section [AA: add this section], where we fit a panel POMP model to the Korevaar et al. (2020) England and Wales measles dataset, which contains over one million observations. While this would normally take over 5 months in `pomp`, `pypomp` can finish the job in 41 hours [AA: revise these numbers as appropriate]. In addition to GPU acceleration, `pypomp` uses JAX to perform gradient descent on differentiable particle filters (Tan et al., 2024), which can further accelerate parameter estimation.

2 POMP Modeling

A POMP model includes a random vector of observed data $Y_{1:N}$, and the observed values are denoted as $y^*_{1:N}$. The data are generated based on the values of the unobserved process, $\{X(t), t_0 \leq t \leq t_N\}$, where $t_{1:N}$ are the times at which $Y_{1:N}$ are observed, t_0 is the time at which the initial state is defined, and $X_n := X(t_n)$. We suppose the model has a joint distribution $f_{Y_{1:N},X_{0:N}}(y_{1:N}, x_{0:N}; \theta)$, where θ is a set of parameters, and Y_n is independent of Y_m, X_m, $m \neq n$, conditional on X_n.

Although evaluating the joint distribution is potentially intractable, we can still implement plug-and-play POMP methodology. For example, particle filtering (Arulampalam et al., 2002) can be used to estimate the marginal likelihood $f_{Y_{1:N}}(y_{1:N};\theta)$, and Iterated Filtering 2 (Ionides et al., 2015) can be used to estimate the maximum likelihood estimate (MLE) of θ, both without an explicit joint distribution or the explicit transition density $f_{X_{n+1}|X_n}(x_{n+1}|x_n;\theta)$. For these methods, it is sufficient to specify a collection of functions associated with the model's conditional distribution densities:

- $X_0 \sim f_{X_0}(x_0;\theta)$: The random number generating process for the initial distribution of the underlying process.
- $X_{n+1} \sim f_{X_{n+1}|X_n}(x_{n+1}|x_n;\theta)$: The random number generating process for the transition density of the underlying process.
- $f_{Y_n|X_n}(y_n|x_n;\theta)$: The measurement density function.

To implement POMP models and methods in `pypomp`, the user can provide data and programmatic descriptions of the above functions as arguments to a `Pomp` object. With these necessary components bundled together, the user can perform inference on the model using `pypomp`'s methods.

3 POMP Inference Methods in `pypomp`

`pypomp` implements several likelihood-based inference methods, all of which leverage the plug-and-play property (Ionides et al., 2006) and JAX-enabled hardware acceleration.

1. **Particle Filter (`pfilter`)**: The standard sequential Monte Carlo (SMC) algorithm for likelihood evaluation and state estimation.
2. **Iterated Filtering (`mif`)**: An implementation of the IF2 algorithm (Ionides et al., 2015) for maximum likelihood estimation. It combines particle filtering with sequential random walk perturbations and a cooling schedule to explore the parameter space.
3. **Automatic Differentiation (`mop` and `train`)**: `pypomp` includes a differentiable particle filter (MOP-α, Tan et al. (2024)) that enables the use of JAX's automatic differentiation. The `train()` method uses these gradients for local optimization.

These algorithms can be run as methods of the `Pomp` class. Users can define their models by providing JAX-compatible functions for each component of the POMP model. For example, the S&P500 model included in the package defines the state transition simulator as follows:

```
import jax
import jax.numpy as jnp
from pypomp.types import StateDict, ParamDict, CovarDict, TimeFloat, StepSizeFloat, RNGKey

def rproc(
    X_: StateDict, theta_: ParamDict, key: RNGKey, covars: CovarDict,
    t: TimeFloat, dt: StepSizeFloat
```

```
):
    V, S = X_["V"], X_["S"]
    mu, kappa, theta = theta_["mu"], theta_["kappa"], theta_["theta"]
    xi, rho, y_prev = theta_["xi"], theta_["rho"], covars["y_prev"]
    dZ = jax.random.normal(key)
    dWs = (y_prev - mu + 0.5 * V) / jnp.sqrt(V)
    dWv = rho * dWs + jnp.sqrt(1 - rho**2) * dZ
    S = S + S * (mu + jnp.sqrt(jnp.maximum(V, 1e-32)) * dWs)
    V = V + kappa * (theta - V) + xi * jnp.sqrt(V) * dWv
    V = jnp.maximum(V, 1e-32)
    return {"V": V, "S": S}
```

Internally, pypomp takes a user-defined function such as this, vectorizes it over the particles and replications of a method (it is prudent to run these methods from multiple values of θ), and JIT compiles it using JAX, achieving high performance by parallelizing computation across GPU cores.

The following example demonstrates a possible inference workflow, expressed in few lines: starting with global exploration via IF2, followed by gradient-based local refinement, and concluding with a final likelihood evaluation.

```
import pypomp as pp
spx_obj = pp.models.spx()
key = jax.random.key(1)
rw_sd = pp.RWSigma( # Random walk standard deviations for IF2
    sigmas={"mu": 0.02, "kappa": 0.02, "theta": 0.02,
            "xi": 0.02, "rho": 0.02, "V_0": 0.1},
    init_names=["V_0"],
)
spx_obj.mif(rw_sd=rw_sd, J=1000, M=10, a=0.5, key=key) # IF2
eta = {name: 0.001 for name in spx_obj.canonical_param_names} # Learning rates
spx_obj.train(J=1000, M=10, eta=eta, optimizer="Adam") # Gradient descent
spx_obj.pfilter(J=1000, reps=10) # Particle filter
```

Each method call generates an object which contains details including log-likelihood estimates, parameter traces, execution time, and the arguments passed to the method. These objects are then appended to `spx_obj.results_history` and can be interfaced with through helper functions, such as the `traces` method, which returns a long-format data frame containing parameter and log-likelihood traces across all method calls.

Methods like `mif` and `train` achieve high performance by internally running JIT-compiled JAX functions from the submodule `pypomp.functional`. The user may import these functions directly to build custom inference procedures.

The open source repository is available at `https://github.com/pypomp/pypomp`, and full documentation is available at `https://pypomp.readthedocs.io`.

4 Discussion

References

Aaron Abkemeier, Jun Chen, Edward Ionides, Jesse Wheeler, and Kevin Tan. Pypomp, 2026.

M.S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, February 2002. ISSN 1053587X. doi: 10.1109/78.978374.

Kidus Asfaw, Joonha Park, Aaron A. King, and Edward L. Ionides. spatPomp: An R package for spatiotemporal partially observed Markov process models. *Journal of Open Source Software*, 9(104):7008, December 2024. ISSN 2475-9066. doi: 10.21105/joss.07008.

Marie Auger-Méthé, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, Joanna Mills Flemming, Anders Nielsen, Giovanni Petris, and Len Thomas. A guide to state-space modeling of ecological time series. *Ecological Monographs*, 91(4):e01470, 2021. doi: 10.1002/ecm.1470. URL `https://doi.org/10.1002/ecm.1470`.

J. C. Blackwood, D. G. Streicker, S. Altizer, and P. Rohani. Resolving the roles of immunity, pathogenesis, and immigration for rabies persistence in vampire bats. *Proceedings of the National Academy of Sciences of the United States of America*, 110(51):20837–20842, December 2013. doi: 10.1073/pnas.1308817110. URL `https://doi.org/10.1073/pnas.1308817110`.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, ake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018.

Carles Bretó. On idiosyncratic stochasticity of financial leverage effects. *Statistics & Probability Letters*, 91:20–26, 2014. doi: 10.1016/j.spl.2014.04.003. URL `https://doi.org/10.1016/j.spl.2014.04.003`.

Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, March 2009. ISSN 1932-6157, 1941-7330. doi: 10.1214/08-AOAS201.

Carles Bretó, Jesse Wheeler, Aaron A. King, and Edward L. Ionides. panelPomp: Analysis of Panel Data via Partially Observed Markov Processes in R, May 2025.

S. J. Fox, M. Lachmann, M. Tec, R. Pasco, S. Woody, Z. Du, X. Wang, T. A. Ingle, E. Javan, M. Dahan, K. Gaither, M. E. Escott, S. I. Adler, S. C. Johnston, J. G. Scott, and L. A. Meyers. Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119, 2022. doi: 10.1073/pnas.2111870119. URL `https://doi.org/10.1073/pnas.2111870119`.

E. L. Ionides, C. Breto, and A. A. King. Inference for nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 103(49):18438–18443, December 2006. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.0603181103.

Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, January 2015. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.1410597112.

Aaron A King, Qianying Lin, and Edward L Ionides. Markov genealogy processes. *Theoretical Population Biology*, 143:77–91, 2022. doi: 10.1016/j.tpb.2021.11.003.

Hannah Korevaar, C. Jessica Metcalf, and Bryan T. Grenfell. Structure, space and size: Competing drivers of variation in urban and rural measles transmission. *Journal of The Royal Society Interface*, 17(168):20200010, July 2020. doi: 10.1098/rsif.2020.0010.

J. A. Jr. Marino, S. D. Peacor, D. B. Bunnell, H. A. Vanderploeg, S. A. Pothoven, A. K. Elgin, J. R. Bence, J. Jiao, and E. L. Ionides. Evaluating consumptive and nonconsumptive predator effects on prey density using field time-series data. *Ecology*, 100(3):e02583, March 2019. doi: 10.1002/ecy.2583. URL `https://doi.org/10.1002/ecy.2583`.

Matthew S. Mietchen, Erin Clancey, Corrin McMichael, and Eric T. Lofgren. Estimating sars-cov-2 transmission parameters between coinciding outbreaks in a university population and the surrounding community, Jan 2024. URL `https://doi.org/10.1101/2024.01.10.24301116`.

Kevin Tan, Giles Hooker, and Edward L. Ionides. Accelerated Inference for Partially Observed Markov Processes using Automatic Differentiation, July 2024.

L. Wen, Y. Yin, Q. Li, Z. Peng, and D. He. Modeling the co-circulation of influenza and covid-19 in hong kong, china. *Advances in Continuous and Discrete Models*, 2024 (1):1–9, 2024. doi: 10.1186/s13662-024-03830-7. URL `https://doi.org/10.1186/s13662-024-03830-7`. Article 32.

5 Appendix

[AA: Include imporant numerical results here? Or refer to the quant repo?]